MedGemma Cardiac CT VQA Training Optimization

Lab210

Open Hackathon Final Presentation | 2026-11-02

OpenACC

More Science, Less Programming

Total presentation time is 5~10 minutes

Lab210



MING JHE TENG
滕明哲
NTUT



Warren Tseng
NVIDIA



CHANG JEN CHIEH
張仁杰
NTUT



Po-Hsuan Tseng
曾柏軒
NTUT

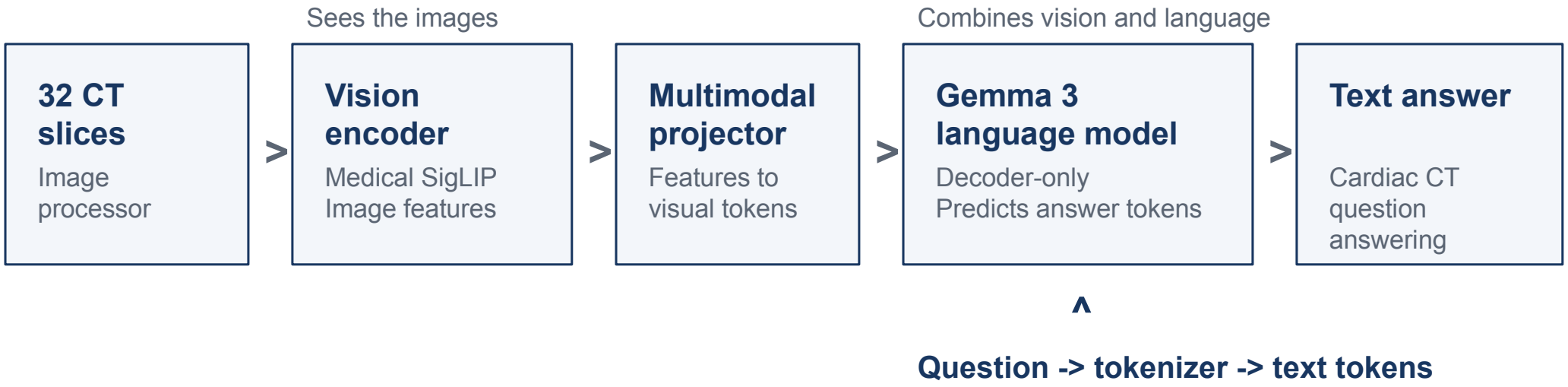


YI CHEN LU
盧翊禎
NTUT

OpenACC More Science, Less Programming
OPEN HACKATHONS
NCHC 國家實驗研究院 國家高速網路與計算中心 National Center for High-performance Computing
NVIDIA

MedGemma 1.5: From CT Images to Answers

Google's 4B medical vision-language model, built on Gemma 3



OUR TASK

Fine-tune with CT images + questions + reference answers using LoRA.

WHY OPTIMIZE?

32 x 256 = 8,192 image tokens before text: vision encoding and attention are costly.

OpenACC More Science, Less Programming
OPEN HACKATHONS
NCHC 國家實驗研究院 國家高速網路與計算中心 National Center for High-performance Computing
NVIDIA

Application Background

32-slice visual question answering with MedGemma 1.5

Input and task

32 CT images + a question -> a short medical answer.

Training workload

10,687 records on Nano5; about 8.3k input tokens, including 8,192 image tokens.

Model and recipe

4B vision-language model; BF16 LoRA, rank 16, alpha 32, dropout 0.05.

Optimization target

Reduce epoch time while preserving the loss definition and checking output quality.

Global batch = 4. This is a systems study, not evidence of clinical readiness.

Optimization Strategy

Isolated probes first; full training and output checks afterward

1. Establish the baseline

Fix data, seed, mask semantics and the comparison environment.

2. Test one factor at a time

Short 32-record probes; separate microstep timing from optimizer updates.

3. Validate the combined recipe

Correct record weighting before accepting the full-epoch candidate.

4. Check the endpoint

Compare adapters on a fixed 119-question subset; reject fast but invalid outputs.

Tested: flex attention, compiled vision, fused kernels, record-weighted loss and adapter A/B.

Results at a Glance

About 4.60x lower observed epoch time across the full migration and optimization path

Moved to Nano5

40h40m -> 20h36m; 2x L40S -> 1x H200.

Production baseline

16h22m32s with the production recipe.

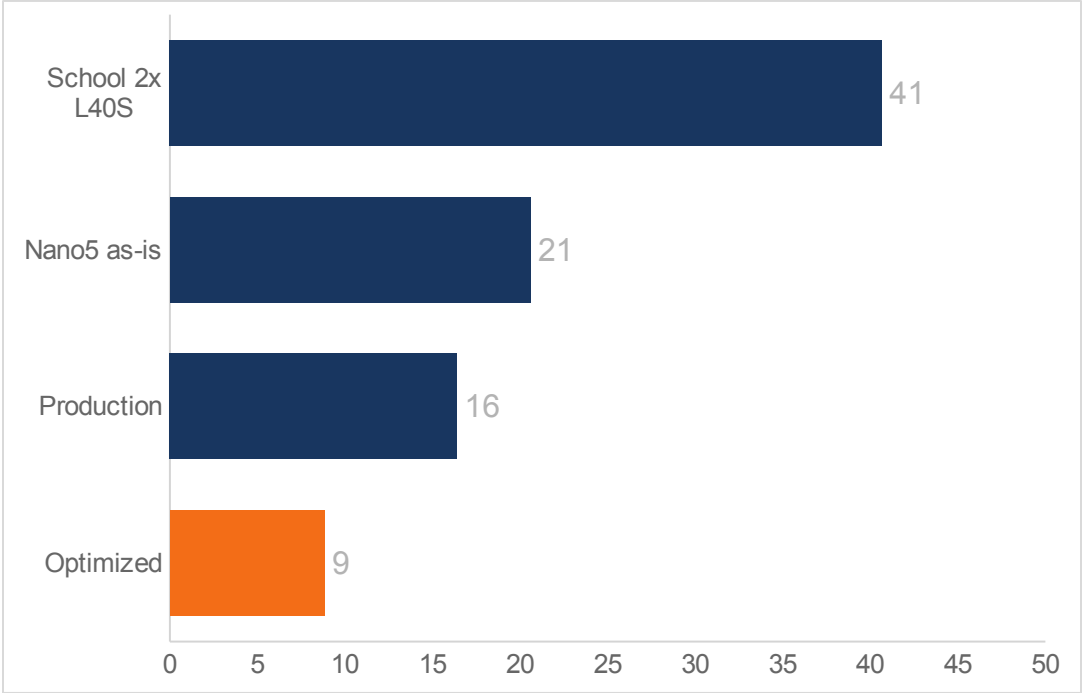
Optimized BF16

8h50m30s with the combined stack; 1.85x faster.

FP8 track: rejected

4h19m34s, but endpoint quality failed.

End-to-end epoch time (hours)



Cross-system observation, not an equal-workload hardware benchmark: 10,280 vs 10,687 records.

Training Time: 32-slice VQA Fine-Tuning

One epoch | MedGemma 1.5 LoRA | school workload: 10,280 records; Nano5: 10,687



	Baseline	Original recipe on H200	Tuned production recipe	Flex + compile + Liger + MB4
Stage speedup:		**1.9741x**	**1.2580x**	**1.8521x**
Time saved:		1,204 min	253.47 min	452.03 min

Total: 4.60x faster | 31h49m30s saved | 78.26% less elapsed time

Stage ratios multiply. Adjacent time savings add; they are not isolated causal contributions.

Per-Component Optimizations

Isolated probes are not additive; the final epoch validates the combined stack

Component	What we changed	Measured
Text attention	SDPA -> flex_attention; preserve mask semantics	5.12 -> 3.72 s -27.34%
Vision tower	torch.compile on the static 32-image batch	5.12 -> 4.60 s -10.16%
Text norm / RoPE / MLP	Liger fused kernels	5.24 -> 4.89 s -6.68%
Batch execution	Microbatch 4 / accum 1; record-weighted loss	Global batch = 4 68-73 GiB in probes
Combined stack	Isolated torch 2.7.1 environment	21 -> 11.9 s/update about -43%

Probe times above are seconds per microstep unless marked per update. Full epoch: -46.01%.

Attention Optimization: Problems and Fixes

The accepted result followed a failed combined run and a loss-weighting correction

Custom attention mask

FA2/3 path did not meet the mask requirements.

Use text-only flex; preserve the image overlay.

Compilation environment

The torch 2.6 path failed.

Isolate torch 2.7.1 and add an SDPA control.

Loss weighting

First combined run deviated by +7.7%.

Restore the per-record mean before retraining.

Revalidation

Corrected probe aligned within about 0.7%.

Train the corrected stack; compare outputs.

Validated: corrected loss weighting and the combined stack. Failed runs are not accepted results.

Result Quality: Similar Subset Metrics

No drop in the reported reference metrics; statistical equivalence remains unproven

Epoch-mean training loss

0.7645 -> 0.7643

Production Optimized

Reference exact match (119 rows)

12.6% -> 14.3%

Production Optimized

Preserved objective

Mask semantics and record weighting retained.

Small loss difference

Epoch mean decreased by 0.026%.

Output agreement is not identity

Adapter outputs match exactly on 88.2% of rows.

Reference contains unchanged

17.65% for both on the same 119 rows.

Optimized adapter: subset evaluation only. No clinical or statistical-equivalence claim.

Transformer Engine FP8: Endpoint Rejected

A faster epoch is not a valid optimization when the resulting model fails evaluation

Stage	Evidence	Decision
TE integration	238 text LoRA base layers; recompute and padding	Epoch completed
Training time	4h19m34s, about 2.04x vs optimized BF16	Speed candidate
119-row endpoint	Exact match 0%; about 66% hit the token cap	Reject
300-step smoke tests	Hybrid, Current and TE BF16 showed long outputs	Root cause unresolved

Accepted result remains BF16: 8h50m30s per epoch

Tested: full-epoch TE FP8; Hybrid / Current FP8 recipes and TE BF16 smoke controls.

Agent Impact: Time to a Validated Result

Evaluate development effort separately from GPU training speed.

Metric	Lab210 evidence / status
Training runtime (measured)	16.38h -> 8.84h per epoch on H200: 1.85x
First probe -> epoch completion	About 20.5h elapsed, reported in the handoff
Including subset output evaluation	About 22.5h elapsed, reported; not human labor
Human effort / manual-only baseline	Not measured; no defensible Agent speedup ratio

Agent contribution: implement probes, diagnose failures and preserve handoffs.

Count retries and recovery costs. Require output checks; reject failed FP8 endpoints.

No manual-only baseline = no measured development-speedup claim.

Method adapted from Hoshino et al., arXiv:2608.13122, Sec. V (2026).

Energy Efficiency Calculator

Baseline	GPU	Select the baseline as GPU or CPU
Baseline GPU Type	8x H200 141GB SXM5	Select the CPU/GPU Type or GPU Node configuration
Baseline GPU # GPUs	1	Select the number of CPU cores or GPUs (within one node)
Final GPU Node	8x H200 141GB SXM5	Select the final GPU Type or GPU Node configuration
Final # GPUs	1	Select the number of GPUs (within one node)
Application Speedup	1.85x	Enter the amount of achieved speedup

Node Replacement	1.85x

GPU NODE POWER SAVINGS			
	Baseline	**Comparison**	
	8x H200 141GB SXM5	8x H200 141GB SXM5	**Power Savings**
Compute Power (W)	19,073	10,298	8,775
Networking Power (W)	1,630	880	750
Total Power (W)	**20,703**	**11,178**	9,525

Node Power efficiency	**1.85x**

ANNUAL ENERGY SAVINGS PER GPU NODE			
	8x H200 141GB SXM5	8x H200 141GB SXM5	**Power Savings**
Compute Power (kWh/year)	167,078	90,210	76,867
Networking Power (kWh/year)	14,277	7,709	6,569
Total Power (kWh/year)	**181,355**	**97,919**	**83,436**

$/kWh	$	0.18
Annual Cost Savings	$	15,018.47
3-year Cost Savings	$	45,055.42

Metric Tons of CO_2	59
Gasoline Cars Driven for 1 year	13
Seedlings Trees grown for 10 years	978

DISCLAIMER

All results shown here are estimates only and should be treated as such as are subject to below seven assumptions. **Do not use the calculator if you are using multinode.**

ASSUMPTIONS

(1) The workload being input will run 24/7/365 on the node in question

(2) When the workload runs on a fraction of a CPU or GPU server, no other bottlenecks occur to stop it from scaling up to occupy the full server

(3) The calculations use TDP for both CPU and GPU. In reality, neither server will run full time at TDP. The comparison here is "worst case CPU" vs. "worst case GPU"

(4) Annual cost savings are operational for electricity only. Capital, personnel, etc are not included

(5) Perfect scaling of the workload to multiple nodes for CPUs

(6) Fractional workload scaling for both CPU and GPU nodes

(7) The GPU machine runs the CPUs a full speed, full power draw

Lab210 | Completed activity calculator screenshot. Formula-based estimate, not measured energy.

Fine-Tuning Workflow and Timing

Separate measured intervals from unmeasured or overlapping stages

Stage	Available timing evidence	Treatment
Data / PNG / processor	Independent duration unknown	Keep 32-slice input
Load / compile / warmup	First sampled microstep: 116.92 s	Not pure compile time
Full training job	31,830 s including job overhead	Accepted combined stack
Validation / checkpoint	Independent duration unknown	Do not add twice
119-question generation	175.5 s for production pass B	Generation time only

Most steady-state updates: about 11.7-12.0 s. Queue, transfer and failed experiments are separate costs.

Conclusions and Next Steps

32-slice system optimization; thesis models and direct128 remain separate work

Accepted H200 improvement

20.6h -> 8.84h per epoch: 2.33x faster.

Observed full-path improvement

About 4.60x across school-to-Nano5 migration and recipe changes.

Quality remains a gate

Subset metrics were similar; FP8 endpoint failed and was rejected.

Next validation

Full-test A/B, patient-level statistics, image-dependence controls and matched TE BF16.

Needed: reproducible GPU access, a stable compiler environment and complete timing logs.

OpenACC More Science, Less Programming
OPEN HACKATHONS
NCHC 國家實驗研究院 國家高速網路與計算中心 National Center for High-performance Computing
NVIDIA

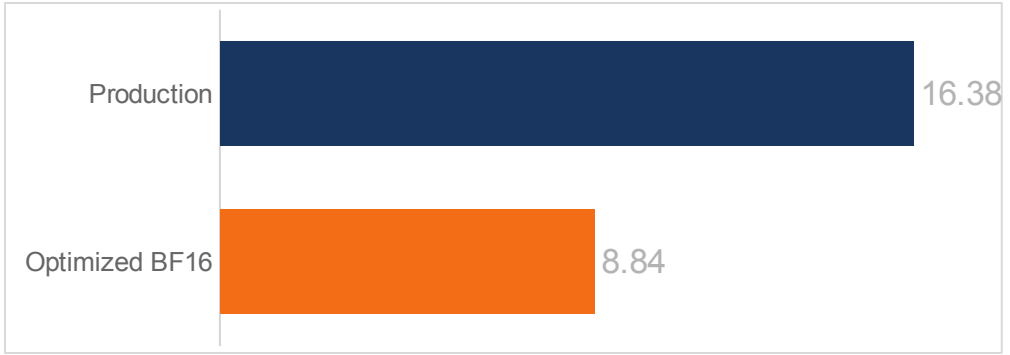
Application Background

Teaching an AI model to answer questions about heart CT scans.

32 images and a question form each example. Faster training enables more experiments within a limited GPU budget.

Main compute: image encoding, attention and model updates.

Representative Result



Same H200: 16.38h -> 8.84h per epoch (1.85x).

Hackathon Objectives and Approach

Reduce training time without changing the learning objective.

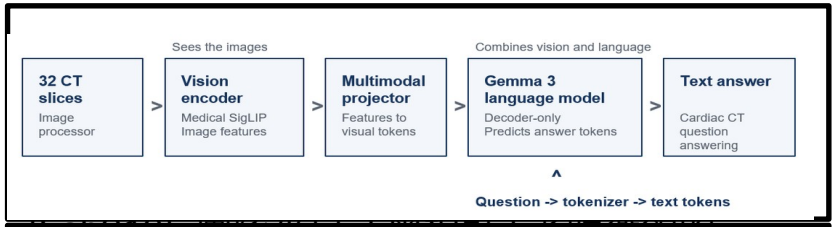
Use PyTorch flex attention, compiled vision execution, Liger fused kernels and microbatch 4.

Run isolated probes, correct loss weighting, then validate the combined stack and outputs.

Technical Accomplishments and Impact

Agent-assisted implementation and validation. Similar subset metrics; FP8 rejected after output checks.

加速 Medgemma1.5 微調



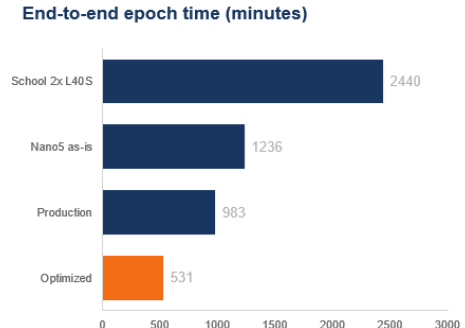
{Lab210} 團隊來自 { 曾柏軒 } 老師帶領的 {210} 實驗室， **AI Agent 將 { 整體加速、模型優化 } 工作流程加速了 {50} 倍； CPU/GPU/DPU 將 {CCTA VLM 模型 } 研究加速了 {4.6} 倍！！**

心臟電腦斷層能提供心臟與冠狀動脈的詳細影像，是研究心血管疾病的重要資料。然而，一次檢查包含大量切片，如何讓人工智慧結合影像與文字，回答與檢查內容相關的問題，是醫療視覺語言模型的重要研究方向。

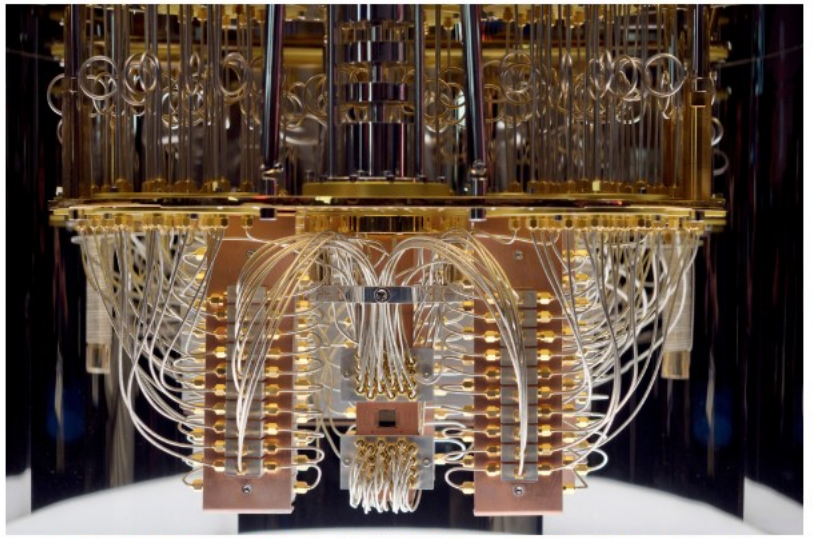
我們使用 MedGemma 1.5 ，將每筆檢查的 32 張切片搭配問題與參考答案進行微調。這類模型需要處理大量影像資訊，訓練時間長，會限制研究者比較不同方法的次數。因此，本次目標是在保留學習目標並檢查回答品質的前提下，提高訓練效率。

透過注意力運算優化、影像編碼器編譯、融合運算與批次調整，同一張 H200 GPU 的單輪訓練時間由 16.38 小時縮短至 8.84 小時，約加速 1.85 倍，每輪節省約 7.5 小時。小型問答測試集的指標未見下降，但仍需更完整的驗證。

加速讓有限的運算資源能支持更多實驗、更快排查問題，也降低每次研究迭代的時間成本。未來可用於比較資料處理與模型設計，逐步建立更可靠的醫療影像問答系統；目前成果仍屬研究用途，不代表已能取代醫師判斷。



量子算法模擬



haofan2023團隊成員來自臺灣大學資工系「洪士灝老師實驗室」，將量子演算法QAOA加速468倍！

— NVIDIA Mentors: Tian Zheng, Frank Lin, Yun-Yuan Wang

量子技術正以驚人的速度發展，預示著我們即將進入量子計算的時代。在這個過程中，量子電路模擬成為一個關鍵工具，它在量子硬體和軟體的開發中扮演著重要的角色，特別是在處理量子程式的編寫和驗證方面。傳統電腦的強模擬能夠獲得完整的量子狀態信息。這使得傳統電腦在構建量子系統方面變得不可或缺，尤其是在當前噪聲較多的中等規模量子（NISQ）時代。

量子近似優化算法（QAOA）是一種常用的量子算法，用於通過近似解來解決組合優化問題。然而，在虛擬量子計算機上執行QAOA對於解決需要大規模量子電路模擬的組合優化問題而言，會遇到模擬速度較慢的問題。團隊使用數學優化來壓縮量子操作，並結合有效的位元操作進一步降低計算複雜性，透過GPU加速最高獲取468倍的加速效果！

Table 1: The elapsed time of 5-level QAOA (unit: second, double).

Qubit	CPU_{Single}	$CPU_{Mutiple}$	CPU_{Cache}	GPU_{Cache}	GPU_{All}
23	29.80	1.28 (23x)	1.28 (63x)	0.24 (120x)	0.06 (**341x**)
24	68.00	3.46 (20x)	3.46 (43x)	0.55 (123x)	0.12 (**382x**)
25	152.52	15.32 (10x)	15.31 (45x)	1.19 (127x)	0.23 (**404x**)
26	330.69	33.83 (10x)	33.83 (56x)	2.60 (126x)	0.56 (**417x**)
27	712.26	72.66 (10x)	72.66 (54x)	5.59 (127x)	1.08 (**427x**)
28	1556.87	156.52 (10x)	156.52 (54x)	11.96 (130x)	2.17 (**445x**)
29	3325.55	335.09 (10x)	335.09 (49x)	25.73 (129x)	4.45 (**451x**)
30	7226.46	718.33 (10x)	718.33 (47x)	55.20 (130x)	9.22 (**468x**)

更多資訊請看：https://github.com/nqobu/nvidia/raw/main/20231207/Team02.pdf

Thank You

Lab210 | Questions & Discussion

OpenACC

More Science, Less Programming